

Hadoop 中的 MapReduce 原理详解

第二讲：数据如何真正进入 Mapper

InputFormat · InputSplit · RecordReader · Mapper

Map 端入口：从 HDFS 文件到 Mapper 的一条记录



核心关系：InputFormat 负责“怎么切 + 怎么读”；InputSplit 描述范围；RecordReader 把范围转换成 Mapper 真正收到的记录。

目标：把“一个 HDFS 文件怎样变成多个 MapTask 的输入”讲透

学习目标

第一讲解决的是“为什么需要 MapReduce”。这一讲开始拆机器内部真正发生的事：当你把一个文件交给 Hadoop 后，系统怎样决定切成几份、启动几个 Mapper、每个 Mapper 从哪里开始读、读到哪里结束，以及为什么 Mapper 收到的是一条条 Key-Value 记录，而不是一大坨原始字节。

- 准确区分 HDFS Block（存储块）与 InputSplit（输入分片），不再把两者当成同一个概念。
- 理解 InputFormat 的两个核心职责：getSplits() 负责“怎么切”，createRecordReader() 负责“怎么读”。
- 理解 RecordReader 为什么存在，以及它怎样把字节区间转成完整记录。
- 知道 TextInputFormat 下 Mapper 的输入通常是 <LongWritable, Text>：Key 是字节偏移量，Value 是一行文本。
- 理解 MapTask 数量为什么通常等于 InputSplit 数量，而不是“直接等于 HDFS Block 数”。
- 能解释跨 Split 的一行文本为什么不会被截成两条错误记录。
- 理解小文件为什么会导致大量 MapTask，以及为什么这会拖慢系统。

先记住总链条

HDFS 文件 -> InputFormat -> InputSplit -> RecordReader -> <K1,V1> -> Mapper。后面所有细节都只是回答：怎么切？怎么读？一条记录长什么样？

0. 和上一讲接起来：Map 阶段不是“直接读文件”

上一讲我们把 Map 理解成“局部并行处理”。但这里有一个隐藏问题：Mapper 并不会自己打开任意文件、自己决定从第几字节读到第几字节。Hadoop 必须先把一个大输入组织成若干个逻辑任务，再为每个任务建立“记录读取器”，最后才调用你的 map() 方法。

Map 端入口：从 HDFS 文件到 Mapper 的一条记录



核心关系：InputFormat 负责“怎么切 + 怎么读”；InputSplit 描述范围；RecordReader 把范围转换成 Mapper 真正收到的记录。

这条链路里最容易混淆的是 InputSplit 与 HDFS Block。很多教材为了入门会说“一块对应一个 Mapper”，这句话在默认场景下经常近似成立，但如果把它当成定义，后面遇到小文件、压缩文件、自定义 InputFormat 时就会出错。

1. 先把 HDFS Block 重新定位：它是“存储层”的概念

1.1 Block 解决的是：一个大文件怎么分布式存

假设你把一个 300 MB 文件上传到 HDFS，默认 BlockSize 为 128 MB。HDFS 会把文件按存储块切开：前两个块各 128 MB，最后一个块约 44 MB。每个 Block 可以有多个副本，分散在不同 DataNode 上。

```
300 MB 文件, BlockSize = 128 MB
Block 0: 0 MB ~ 128 MB
Block 1: 128 MB ~ 256 MB
Block 2: 256 MB ~ 300 MB (约 44 MB)
```

Block 的主要目标是存储管理和容错：NameNode 记录每个块在哪些 DataNode 上；DataNode 保存实际块数据；副本机制在节点故障时保护数据。这里还没有决定“要启动几个 Mapper”。

关键

HDFS Block 回答的是“数据物理上怎么存、存在哪里”；它属于 HDFS 存储层。

2. InputSplit：它是“计算层”的逻辑任务边界

2.1 InputSplit 不保存真正的数据

InputSplit 可以理解成一张“任务说明单”：告诉某个 MapTask——你要处理哪个文件、从哪个位置开始、逻辑长度是多少、最好去哪些节点附近执行。真正的数据仍然在 HDFS 中。

常见的 FileSplit 会携带类似下面的信息：

```
Path    = hdfs://.../logs.txt
Start   = 134217728           # 从文件的哪个字节位置开始
Length  = 134217728           # 这个逻辑分片大约多长
Hosts   = [node2, node5]      # 哪些节点保存相关 HDFS Block，便于数据本地性调度
```

类比

Block 像“仓库里的货物箱”；InputSplit 像“派工单”。派工单可以参考货物箱的位置来安排工作，但派工单本身不是货物箱。

2.2 为什么默认情况下 Split 常常和 Block 差不多大？

因为这样最有利于数据本地性。一个 Split 如果主要落在一个 HDFS Block 内，调度器就更容易把 MapTask 放到拥有该 Block 副本的节点上，本地读取大部分数据，减少网络传输。FileInputFormat 默认计算 Split 大小时，会参考 BlockSize。一个常见的抽象公式是：

$$\text{splitSize} = \max(\text{minSize}, \min(\text{maxSize}, \text{blockSize}))$$

在没有人为调整 minSize / maxSize 时，splitSize 往往接近 blockSize，所以才会出现“一个 Block 对应一个 InputSplit，一个 InputSplit 对应一个 MapTask”的常见现象。注意：这是默认策略产生的结果，不是概念定义。

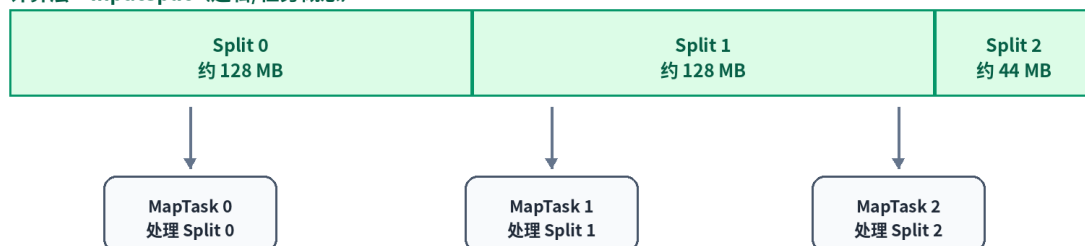
HDFS Block 与 InputSplit：一个负责存，一个负责算

示例：一个 300 MB 文本文件，HDFS BlockSize = 128 MB

存储层：HDFS Block（物理/存储概念）



计算层：InputSplit（逻辑/任务概念）



在默认 TextInputFormat / FileInputFormat 且文件可切分时，Split 大小常接近 Block 大小，因此常见“1 个 Block $\approx$ 1 个 Mapper”；但这不是定义，更不是

2.3 Block 与 Split 的精确区别

维度	HDFS Block	InputSplit
所属层次	HDFS 存储层	MapReduce 计算输入层
目的	把大文件分块存储、复制、容错	把输入划成若干 MapTask 的逻辑工作量
是否真的保存数据	是，块数据存于 DataNode	否，只描述范围和位置信息

由谁决定	HDFS / 文件系统配置	InputFormat (常见为 FileInputFormat)
是否必须等大	不适用	不必; 默认情况下常接近 BlockSize
与 Mapper 的关系	间接影响数据位置与常见 Split 大小	通常 1 个 Split 对应 1 个 MapTask
能否自定义	可配置 BlockSize	可通过 InputFormat / split 参数改变

3. InputFormat: MapReduce 输入端的“总设计师”

3.1 InputFormat 的两个核心问题

InputFormat 不只是“文件格式”。从 MapReduce 框架的角度，它至少要回答两个核心问题：

1. 怎么把整个输入划分成若干个 InputSplit? 这决定 Map 任务的逻辑并行度。
2. 每个 InputSplit 里面，怎样不断读出一条条 <Key,Value> 记录? 这由 RecordReader 完成。

```
InputFormat
|-- getSplits(job)          -> List<InputSplit>
|-- createRecordReader(split) -> RecordReader<K,V>
```

所以可以把 InputFormat 记成一句话：“它定义输入怎样被切开，以及切开后怎样被解释成记录。”

3.2 最常见的 TextInputFormat 做了什么?

TextInputFormat 是 Hadoop MapReduce 最常见的输入格式之一。它通常把“文本中的一行”当成一条记录。它继承 FileInputFormat 的文件切分逻辑，再使用行记录读取器把字节内容解析成完整行。

输入格式	记录边界通常是什么	Mapper 输入示例
TextInputFormat	一行文本	<字节偏移量, 该行文本>
KeyValueTextInputFormat	一行，再按分隔符切 Key/Value	<字段 1, 字段 2>
NLineInputFormat	按指定行数组织 Split	仍以行为记录
SequenceFileInputFormat	Hadoop SequenceFile 中的记录	文件中原生 Key/Value

不要混淆

InputFormat 决定“输入记录的解释方式”和“Split 生成方式”；它不是 Mapper，也不负责你的业务计算。

4. Mapper 数量到底是怎么决定的？

4.1 最重要的近似关系

MapTask 数量 $\approx$ InputSplit 数量

对典型 MapReduce Job 来说，一个 InputSplit 通常由一个 MapTask 处理。因此，真正直接决定 Mapper 数量的是 InputFormat 产生了多少个 Split。HDFS Block 只是通过 blockSize、位置信息等“间接影响” Split。

4.2 300 MB 文件的例子

仍以 300 MB 文本文件、128 MB BlockSize、默认可切分 TextInputFormat 为例。若没有特殊配置，FileInputFormat 往往产生约 3 个 Split，于是通常会启动约 3 个 MapTask。

```
Split 0: start=0 MB,   length≈128 MB  -> MapTask 0
Split 1: start=128 MB, length≈128 MB  -> MapTask 1
Split 2: start=256 MB, length≈44 MB   -> MapTask 2
```

这里的“ $\approx$ ”很重要：记录边界、压缩格式、自定义 InputFormat、CombineFileInputFormat、split min/max 参数都会改变实际情况。

4.3 Mapper 越多越好吗？

不是。更多 Mapper 可以增加并行度，但每个 MapTask 都有调度、Container 分配、进程/JVM 启动、状态跟踪、临时文件管理等固定成本。如果 Split 太小，系统可能把大量时间花在“启动任务”而不是“处理数据”上。

底层权衡

Split 太大：并行度不足，少数任务很重；Split 太小：任务数量爆炸，调度与启动开销过高。

MapReduce 的并行度设计，本质上是在“任务粒度”和“固定开销”之间做平衡。

5. RecordReader：真正把字节变成 Mapper 能吃的记录

5.1 为什么不能让 Mapper 直接处理 InputSplit?

InputSplit 只是一段逻辑范围，例如“从 128 MB 位置开始，长度约 128 MB”。但 Mapper 希望得到的是业务可理解的一条记录，例如“一行日志”。中间必须有人把原始字节流解释成记录，这个人就是 RecordReader。

```
InputSplit: [start=134217728, length=134217728]
```

|

v

RecordReader 逐条返回：

<K1, V1>

<K1, V1>

<K1, V1>

|

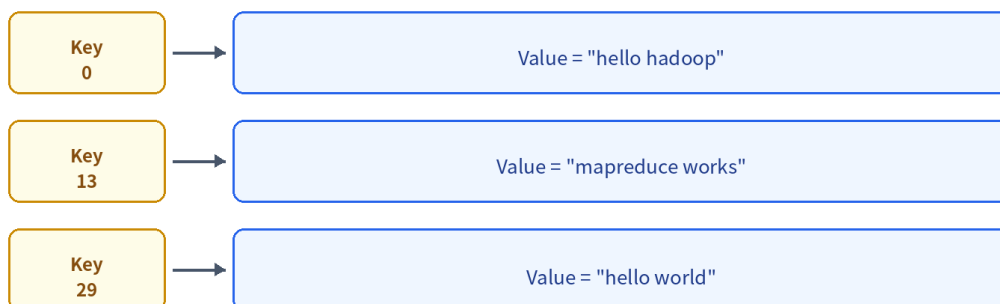
v

Mapper.map(K1, V1, context)

框架会不断调用 RecordReader 的 nextKeyValue()，每成功读出一条记录，就取 currentKey 和 currentValue，再调用一次 Mapper 的 map()。因此，从 Mapper 的视角看，它只是“一条一条地收到记录”。

5.2 TextInputFormat 下，Key 和 Value 分别是什么？

TextInputFormat 给 Mapper 的 Key 为什么是“字节偏移量”？



Key 不是“行号”，而是这条记录在文件中的起始字节位置（LongWritable）；Value 是该行文本（Text，通常不含换行符）。

默认 TextInputFormat 的记录读取器通常给出：Key = 当前行起始位置相对于文件开头的字节偏移量；Value = 当前行文本。对应 Java 类型通常是 LongWritable 和 Text。

```
public static class MyMapper
```

```

extends Mapper<LongWritable, Text, Text, IntWritable> {

    protected void map(LongWritable key, Text value, Context context) {
        // key   : 这一行在文件中的起始字节位置
        // value : 这一整行文本
    }
}

```

初学者经常把 key 当“行号”。这不准确。它是字节偏移量，所以如果前一行更长，下一行的 Key 增量也更大；不同字符编码还会影响“字符数”和“字节数”的对应关系。

6. 最容易卡住的问题：Split 边界把一行切成两半怎么办？

6.1 Split 是按字节范围切，记录却必须保持完整

假设一个 Split 的结束位置正好落在某一长行中间。如果 Hadoop 机械地在字节边界停止，那么同一行会被拆成两条残缺记录，业务语义就错了。TextInputFormat 的 RecordReader 会专门处理这个问题。

TextInputFormat 如何处理“某一行跨越 Split 边界”

假设 Split 边界落在一行文本中间。逻辑切分是按字节范围，记录读取必须按“完整行”。



6.2 两侧各怎么处理？

- 前一个 Split：如果已经开始读取某一条记录，会继续读取到该记录的结束位置（例如换行符），即使物理读取位置略微超过 Split 的逻辑末尾。
- 后一个 Split：如果 splitStart 不是文件开头，而它落在上一条记录中间，RecordReader 会避免把“上一条记录的后半截”再当成新记录，通常从下一条完整记录开始返回。

因此，InputSplit 的 start/length 是“任务逻辑边界”，而 RecordReader 负责尊重“记录语义边界”。这正是为什么 InputSplit 和 RecordReader 必须分成两个抽象。

一句话

Split 管“工作量范围”，RecordReader 管“记录完整性”。

7. 一条记录从 HDFS 到 map() 的完整生命周期

现在把过程完整串起来。假设你运行一个 WordCount Job，输入目录中有一个大型文本文件。

3. 客户端提交 Job，并把输入路径等配置交给 MapReduce。
4. InputFormat 扫描输入文件，根据文件长度、BlockSize、split 配置、文件是否可切分等生成 InputSplit 列表。
5. 每个 InputSplit 对应一个 MapTask 的逻辑输入；YARN 尽量把该任务调度到拥有相关 HDFS Block 的节点附近。
6. MapTask 创建 RecordReader，并把它初始化到对应 Split。
7. RecordReader 从 HDFS 读取字节，并解析出第一条完整记录。
8. 框架取得 key/value，调用一次 mapper.map(key, value, context)。
9. RecordReader 继续 nextKeyValue()，框架继续调用 map()，直到这个 Split 的记录读完。
10. Mapper 输出中间 Key-Value，下一讲再进入缓冲区、Spill、Sort、Partition。

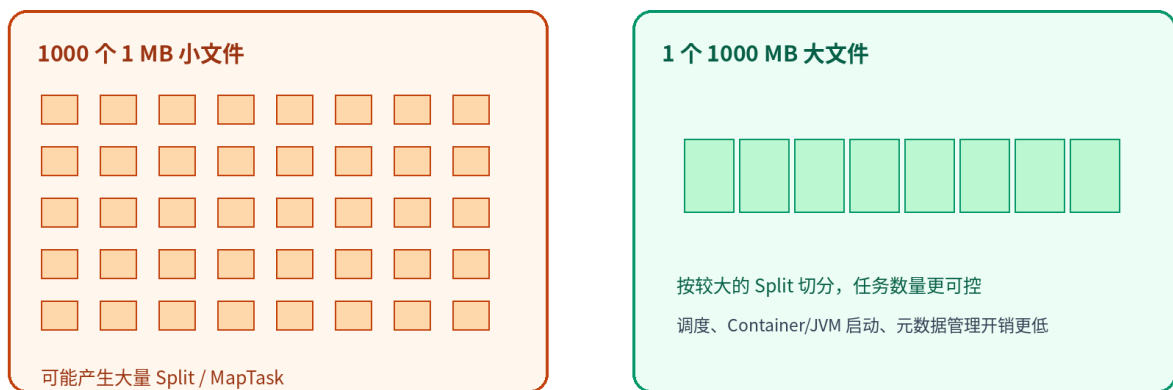
请建立这个机械感

Mapper 并不是“自己循环读文件”。循环读取记录、调用 map() 的外层框架由 Hadoop 控制；你写的 map() 更像“给我一条记录，我告诉你怎么处理这一条”。

8. 小文件问题：为什么 1000 个 1 MB 文件很讨厌？

MapReduce 的默认文件输入机制常以文件和 Split 为任务组织单位。大量小文件很容易产生大量很小的 Split，从而生成大量 MapTask。每个任务即使只处理 1 MB，也仍然有固定的调度和启动成本。

小文件问题：文件很小，不代表任务开销也很小



8.1 小文件的两层代价

- HDFS 层：NameNode 要维护大量文件和 Block 的元数据。文件越碎，元数据对象越多。
- MapReduce 层：大量小 Split 造成大量 MapTask，Container/JVM 启动和调度开销占比变高。

8.2 常见思路

工程上常见的解决思路包括：上游先合并小文件；使用 SequenceFile/Avro/Parquet 等更适合批量存储的容器格式；在 MapReduce 输入端使用 CombineFileInputFormat，把多个小文件组合到更少的逻辑 Split 中。

注意

“HDFS 适合大文件”不是因为它不能存小文件，而是因为海量小文件会放大元数据与任务管理开销。

9. 一个重要边界条件：压缩文件不一定能切

Split 能否任意产生，还取决于文件格式是否支持“从中间位置开始解码”。普通未压缩文本通常可以按字节范围切分；但某些压缩格式存在流式依赖，从文件中间跳进去无法正确恢复解码状态。

情况	典型理解	对 Mapper 数量的影响
普通未压缩文本	通常可切分	大文件可以产生多个 Split / Mapper
gzip 文本	通常不可切分	一个很大的 .gz 文件常只能形成一个主要 Split，Map 并行度受限
支持可切分的容器/压缩格式	存在同步点或块边界	可以从合法边界并行读取

这说明“文件有几个 HDFS Block，就一定有几个 Mapper”明显不成立。一个 10 GB 的 gzip 文件可以跨很多 HDFS Block 存储，但因为输入格式不可切分，MapReduce 可能无法按这些 Block 并行建立多个独立文本解码任务。

10. 初学者最常见的 12 个疑问

Q1: InputSplit 是 HDFS 里真实存在的文件吗？

不是。它主要是描述输入范围和位置信息的逻辑对象。

Q2: 一个 Block 必然对应一个 Split 吗？

不必然。默认 FileInputFormat 场景中常接近一一对应，但参数、文件格式和自定义 InputFormat 都能改变它。

Q3: 一个 Split 必然对应一个 Mapper 吗？

典型 MapReduce 执行模型中，通常是一个 InputSplit 交给一个 MapTask 处理。

Q4: Mapper 数量能不能直接设置成 100？

经典 MapReduce 中不能把“map 数量”当作绝对命令；它主要由 InputFormat 的 Split 数决定。某些旧参数更多是提示或影响切分。

Q5: RecordReader 是我写的吗？

大多数常规任务直接使用 InputFormat 自带的 RecordReader；只有特殊记录格式才需要自定义。

Q6: TextInputFormat 的 Key 是行号吗？

不是，是该记录起始处的字节偏移量。

Q7: 如果一行比 128 MB 还长怎么办？

RecordReader 仍会尝试读取完整记录，因此单条超长记录可能突破 Split 的名义边界并造成内存/性能问题。

Q8: Split 边界跨行会不会丢数据？

正常实现不会。RecordReader 协调相邻 Split，避免丢记录或把同一记录重复作为两条返回。

Q9: 为什么 Split 还要记录 hosts？

为了数据本地性调度，让 YARN 尽量把 MapTask 放到保存相关 HDFS Block 的节点附近。

Q10: 多个小文件能不能共用一个 Mapper？

可以通过 CombineFileInputFormat 等机制把多个文件组合进一个逻辑 Split。

Q11: Mapper 处理的是 Block 还是 Split？

从 MapReduce 任务语义看，Mapper/MapTask 处理的是 InputSplit；Block 是底层数据来源。

Q12: 为什么 Hadoop 不直接规定“每 128 MB 一个任务”？

因为不同输入格式有不同记录结构和可切分性，计算逻辑不能被物理 BlockSize 完全绑死。

11. 计算题：你现在应该能估算 Mapper 数量了

例 1: 一个 1 GB 文本文件

假设 HDFS BlockSize = 128 MB，文件未压缩，使用默认 TextInputFormat，没有人为改变 split min/max。粗略估算：

1 GB = 1024 MB
 1024 / 128 = 8
 因此通常约产生 8 个 InputSplit -> 约 8 个 MapTask

例 2：同样 1 GB，但由 1024 个 1 MB 小文件构成

如果按普通 FileInputFormat 逐文件形成 Split，不能简单把 1024 个小文件自动拼成 8 个 128 MB Split；可能产生数量非常大的 MapTask。总数据量一样，任务组织开销却完全不同。

例 3：一个 1 GB 的 gzip 文件

即使它在 HDFS 中占据多个 Block，由于 gzip 通常不可切分，MapReduce 文本输入往往不能从压缩流中间独立开始解码，因此通常不会得到 8 个真正并行的文本 Split。

迁移规律

估算 Mapper 数量时，不要只问“数据多大、Block 多大”。至少要同时问：有几个文件？InputFormat 是什么？文件是否可切分？split 参数是什么？

12. 本讲最终心智模型：把“存储边界”和“记录边界”分开

层次	它关心什么	核心对象
存储层	数据怎样分块、复制、放在哪些 DataNode	HDFS Block
任务切分层	一次 MapTask 应负责多大逻辑范围	InputSplit
记录解析层	怎样从字节范围取出完整业务记录	RecordReader
业务计算层	拿到一条 <K,V> 后做什么	Mapper.map()

把这四层分开以后，很多看似复杂的问题会自动变清楚。例如“跨 Block 的一行怎么处理？”其实不是 Mapper 的职责；“Mapper 为什么有 3 个？”要先看 InputSplit；“为什么任务被安排到 node2？”要看 Split 的数据位置信息与 HDFS Block 副本。

一句话总结

Block 决定数据怎么存；Split 决定任务怎么分；RecordReader 决定记录怎么读；Mapper 决定一条记录怎么处理。

13. 迁移练习：不写代码，先判断机制

练习 1：一个 500 MB 未压缩日志文件，BlockSize=128 MB。默认 TextInputFormat 下，Mapper 大约几个？

参考答案

大约 4 个（128+128+128+约 116 MB），但这是默认可切分场景下的估算。

练习 2：一个 500 MB gzip 日志文件跨了 4 个 HDFS Block，能否据此断定有 4 个 Mapper？

参考答案

不能。gzip 通常不可切分，HDFS Block 数不等于文本输入的 Split 数。

练习 3：Split 0 的结束位置落在一条日志中间，这条日志会不会被切成两个 value？

参考答案

正常 TextInputFormat/LineRecordReader 会维护完整记录，不会把它机械切成两个 Mapper 输入 value。

练习 4：为什么 10 GB 数据放成 10000 个 1 MB 文件，常比少量大文件更难处理？

参考答案

因为 HDFS 元数据对象、Split/MapTask 数、调度和启动开销都会膨胀。

附录：本讲术语速查

术语	通俗解释
HDFS Block	HDFS 把大文件分布式保存时使用的存储块。
InputFormat	定义输入怎样切成 Split，以及怎样创建 RecordReader。
InputSplit	一个 MapTask 的逻辑输入范围和位置描述，不保存真实数据。
FileSplit	文件输入场景中常见的 InputSplit 实现，含 path/start/length/hosts。
RecordReader	把 Split 对应的字节流解析成一条条 <Key,Value> 记录。
TextInputFormat	常见文本输入格式，通常一行作为一条记录。

LongWritable	Hadoop 可序列化的长整型类型；TextInputFormat 常用它表示字节偏移量。
Text	Hadoop 的字符串/字节序列类型；TextInputFormat 常用它承载一行文本。
Splittable	输入文件能否从多个独立位置开始解析，从而形成并行 Split。
Data Locality	尽量让 MapTask 靠近保存其输入数据的节点运行。

下一讲预告

下一讲进入 Mapper 输出端：Mapper 产生的中间结果为什么不会直接写 HDFS？它们先进入哪里？什么是内存缓冲区、Spill（溢写）、Map 端 Sort、Partition？然后我们会为第四讲的 Shuffle 做铺垫。那一讲开始，你会看到 MapReduce 性能瓶颈真正出现在哪里。